

BuildingImprovingDocumentRAGSystem

May 19, 2026

1 Lab 8: Building and Improving a Document RAG System

```
[1]: import os
from openai import OpenAI

from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

from IPython.display import display, Markdown

client = OpenAI()
```

Throughout this unit, you have explored how RAG systems work and built the individual pieces: embeddings and vector similarity, document loaders, chunking strategies, vector indexes, RAG chains, and query transformation techniques. In this lab, you will assemble those pieces into a complete RAG application for an outdoor gear company, then improve its retrieval quality by adding a query rewriting step on top.

You will complete the following tasks:

1. Load and chunk the knowledge base:
 - Load a customer support corpus from a single text file
 - Choose chunking parameters and justify your choice in writing
2. Build the vector index:
 - Initialize an embedding model and a Chroma vector store
 - Inspect retrieval quality on a test query using similarity scores
3. Write the RAG instruction prompt:
 - Write a prompt template that grounds the model in retrieved context and handles unknowns
4. Assemble the baseline RAG chain:
 - Connect the retriever, prompt, LLM, and output parser using the pipe operator
 - Test the chain on three different kinds of customer queries
5. Add query rewriting and compare:
 - Write a query rewriting prompt that expands short customer queries

- Compare baseline answers to rewritten-query answers on the same three queries
6. Analysis:
 - Evaluate the pipeline’s behavior, limitations, and business implications
 7. AI Reflection:
 - Reflect on how you used (or did not use) AI tools during this lab

Important: All graded cells that require you to enter code will contain regions marked with `# YOUR CODE HERE` and `# END OF YOUR CODE`. Between these lines, you’ll find the line `raise NotImplementedError`(“Your code is missing.”), like so:

```
# YOUR CODE HERE
raise NotImplementedError("Your code is missing.")
# END OF YOUR CODE
```

These markers indicate exactly where you should enter your answer. Replace the line `raise NotImplementedError`(“Your code is missing.”) with your code.

1.1 Business Context

Read through the scenario below. You will be putting yourself in the shoes of a junior ML engineer (MLE) at Marlowe & Finch, where you have been asked to prototype a customer support assistant for the company’s website.

1. Company and Context Marlowe & Finch is a small outdoor gear company based in Boulder, Colorado. The company was founded in 2017 by two former trail crew leads who got tired of gear that looked great in the store and fell apart on the trail. Today Marlowe & Finch designs lightweight, three-season backpacking gear for weekend backpackers and thru-hikers, and sells through its own website and a handful of independent outdoor retailers.

The customer base is mostly enthusiastic but not expert. Many customers are buying their first real piece of backcountry gear and have a lot of practical questions before they pull the trigger on a \$400 tent.

2. Business Challenge Marlowe & Finch’s customer support team currently answers most questions by hand, working from an internal knowledge base of product specs, warranty terms, and returns/shipping policies. Response times have been slipping as the volume of presale questions grows, and customers who do not get a quick answer often abandon their carts. The team has been asked to build a prototype assistant that can answer common product and policy questions on the website, using the same knowledge base the human agents reference today.

3. Business Goal The goal is to build a working RAG (Retrieval-Augmented Generation) prototype that can take a customer’s question, retrieve the most relevant passages from the Marlowe & Finch knowledge base, and produce a grounded answer. If the prototype performs well, it would be deployed as a first-line assistant on the website. Customer support agents would handle anything the assistant cannot.

4. Your Role and Task You have just joined Marlowe & Finch as a junior MLE on the Customer Experience team. The team has handed you the customer-facing knowledge base as a single text file and asked you to build and test a prototype RAG assistant. Your job is to make

the chunking, retrieval, and prompting decisions, then evaluate whether the prototype produces answers the customer support team would be comfortable putting in front of customers.

This work requires judgment at every stage. The chunking parameters you pick determine what the retriever can find. The retrieval setup determines what context the model sees. The instruction prompt determines whether the model stays grounded in the retrieved context or wanders into territory the knowledge base does not cover. A small change in any of these stages can meaningfully change the answers customers receive.

5. Technical Focus in This Lab This lab focuses on building and improving a document RAG pipeline:

- **Document Loading and Chunking** – Loading a source document and splitting it into focused chunks with `RecursiveCharacterTextSplitter`.
- **Vector Indexing and Retrieval** – Embedding chunks with OpenAI embeddings and storing them in a Chroma vector store for similarity search.
- **Prompt Engineering for RAG** – Writing an instruction prompt that grounds the model in retrieved context and handles questions the knowledge base cannot answer.
- **Chain Composition with LangChain** – Using the pipe operator to assemble a complete RAG chain.
- **Query Transformation** – Adding a query rewriting step in front of the retriever to improve retrieval on short, vague queries.

1.2 Part 1. Load and Chunk the Knowledge Base

Marlowe & Finch has provided you with a single text file containing the customer-facing knowledge base. It is stored at `data/marlowe_knowledge_base.txt` and includes the company’s “About” content, specs for three products, warranty terms, returns and shipping policies, and an FAQ.

Before you can search this knowledge base with embeddings, you need to load it and split it into chunks. You practiced both steps in the LangChain primer and the chunking strategies activity.

Task: Use `TextLoader` to load the file at `data/marlowe_knowledge_base.txt`. Save the result to a variable called `documents`. Then print how many documents you loaded and the total character length, so you can sanity-check that the file was read correctly.

Tip: `TextLoader(path).load()` returns a list of `Document` objects.

```
[2]: ### BEGIN SOLUTION

# --- Solution (pattern from LangChain Primer and Chunking Strategies_
↳activities) ---
loader = TextLoader("data/marlowe_knowledge_base.txt")
documents = loader.load()

print(f"Loaded {len(documents)} document(s).")
print(f"Total length: {len(documents[0].page_content):,} characters")

### END SOLUTION
```

Loaded 1 document(s).
Total length: 12,430 characters

Task: Print the first 1,500 characters of the document so you can see what kind of content you are working with. You will use this view to inform your chunking choices in the next step.

Tip: You can slice a string with `text[:1500]`. The full document content lives in `documents[0].page_content`.

```
[3]: ### BEGIN SOLUTION  
# --- Solution (.page_content access pattern from LangChain Primer activity) ---  
print(documents[0].page_content[:1500])  
### END SOLUTION
```

```
=====
```

MARLOWE & FINCH CUSTOMER SUPPORT KNOWLEDGE BASE
Last updated: Spring 2025

```
=====
```

Welcome to the Marlowe & Finch customer support knowledge base. This document is the internal source of truth our support team uses to answer customer questions. It covers our current product lineup, warranty terms, returns and shipping policies, and frequently asked questions.

```
=====
```

ABOUT MARLOWE & FINCH

```
=====
```

Marlowe & Finch is a small outdoor gear company based in Boulder, Colorado. We design backcountry equipment for weekend backpackers and thru-hikers, with a focus on lightweight three-season gear that holds up to real use. Our products are sold through our website and a handful of independent outdoor retailers. We do not currently sell through Amazon, REI, or other large marketplaces.

We were founded in 2017 by two former trail crew leads who got tired of gear that looked great in the store and fell apart on the trail. Every product in our lineup is field-tested by our own staff before it ships.

```
=====
```

PRODUCT CATALOG

```
=====
```

```
-----
```

Trailhead 2 Tent

```
-----
```

The Trailhe

Now that you have seen the content, take a moment to think about its structure. It mixes brand story, product specs, policy language, and FAQ entries. A chunk that mixes a product spec with an unrelated policy paragraph would be a noisy hit for either kind of query. A chunk that is too small might cut a policy sentence in half so that neither chunk contains a complete answer.

You will now split the document into chunks. The chunking strategies activity covered the trade-offs between small, medium, and large chunks, and the role of overlap.

Task: Create a `RecursiveCharacterTextSplitter` with `chunk_size` and `chunk_overlap` values of your choosing. Then call `.split_documents()` on `documents` to produce the chunks. Save the result to a variable called `chunks`.

After running your splitter, print the total number of chunks and the average chunk length so you can sanity-check your choice.

```
[4]: ### BEGIN SOLUTION
# --- Solution (RecursiveCharacterTextSplitter usage from Chunking Strategies
    ↪activity) ---
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=75
)
chunks = splitter.split_documents(documents)

avg_chunk_size = sum(len(c.page_content) for c in chunks) / len(chunks)
print(f"Total chunks: {len(chunks)}")
print(f"Average chunk length: {avg_chunk_size:.1f} characters")
### END SOLUTION
```

Total chunks: 35

Average chunk length: 358.6 characters

Task: In the markdown cell below, answer the following:

1. What `chunk_size` and `chunk_overlap` did you choose?
2. Why did you pick those values for this particular knowledge base? Refer to something specific you saw in `documents[0].page_content`.
3. What is one risk of your choice (for example, what kind of customer question might it handle poorly)?

<Double click this Markdown cell to make it editable, and record your findings here.>

1.3 Part 2. Build the Vector Index

Now that you have chunks, you will embed them and store them in a vector database so the retriever can find the most relevant chunks for any customer query. You did this same pipeline in the Build a Vector Index activity.

Task: Initialize an `OpenAIEmbeddings` object using the `text-embedding-3-small` model. Save it to a variable called `embeddings`. Then build a Chroma vector store from your chunks using `Chroma.from_documents()`. Save the result to a variable called `vectorstore`.

```
[5]: ### BEGIN SOLUTION

# --- Solution (OpenAIEmbeddings + Chroma.from_documents from Build a Vector
↳Index activity) ---
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings
)
### END SOLUTION
```

Task: Create a retriever from your vector store using `vectorstore.as_retriever()`. Configure it to use similarity search and to return the top 4 chunks. Save it to a variable called `retriever`.

Tip: `as_retriever()` accepts `search_type` and `search_kwargs` arguments. The `k` value goes inside `search_kwargs`.

```
[6]: ### BEGIN SOLUTION

# --- Solution (as_retriever pattern from Build a Vector Index and Build Your
↳First RAG System activities) ---
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}
)
### END SOLUTION
```

Before you build the full RAG chain, let's inspect what the retriever returns for a real customer query, along with the similarity scores. The cell below uses `similarity_search_with_score()` so you can see how close each retrieved chunk actually is to the query.

```
[7]: # Do not remove or edit this cell

test_query = "what's your return policy"

scored_results = vectorstore.similarity_search_with_score(test_query, k=4)

print(f"Query: \"{test_query}\"")
print(f"\nTop 4 retrieved chunks (lower distance = more similar):\n")
for i, (doc, score) in enumerate(scored_results, 1):
    preview = doc.page_content[:200].replace("\n", " ")
    print(f"--- Chunk {i} (distance: {score:.4f}) ---")
    print(f"{preview}...\n")
```

Query: "what's your return policy"

Top 4 retrieved chunks (lower distance = more similar):

--- Chunk 1 (distance: 0.8323) ---

```
===== RETURNS AND SHIPPING  
POLICY =====  
-----...
```

--- Chunk 2 (distance: 0.9207) ---

To start a return: 1. Email returns@marloweandfinch.com with your order number.
2. We will email you a prepaid return label within one business day. 3. Pack the item in its original packaging and drop...

--- Chunk 3 (distance: 1.1308) ---

To qualify as "unused," items must be: - In original packaging with all tags attached - Free of dirt, pet hair, scent, and any signs of outdoor use - Free of modifications (no aftermarket patches, sew...

--- Chunk 4 (distance: 1.2196) ---

Q: What's your sustainability policy? A: We are a small company and try to be honest about what we do and do not do on sustainability. Our shell fabrics are bluesign-approved, which means the mills th...

Task: Look at the output above and think about how well the retriever is working. In the markdown cell below, answer the following:

1. Are the top retrieved chunks actually about the return policy, or are some of them off-topic?
2. What do the distance scores tell you? Is there a clear gap between the top result and the bottom result, or are they similar?
3. If you saw an off-topic chunk in the top 4, what do you think pulled it in? If all four chunks were on-topic, would your answer change if a customer asked something more specific, like "can I return a tent I used once"?

<Double click this Markdown cell to make it editable, and record your findings here.>

1.4 Part 3. Write the RAG Instruction Prompt

You have a retriever. Now you need an instruction prompt that tells the LLM what to do with the retrieved context. You wrote one of these in the Build Your First RAG System activity. In that activity, the prompt was for a generic question-answering assistant. Here you will write one tailored to Marlowe & Finch's customer support tone and to the specific risks of running a customer-facing assistant.

A good RAG prompt for this use case should:

1. Position the model as a customer support assistant for Marlowe & Finch
2. Tell the model to answer using only the retrieved context
3. Tell the model exactly what to say when the context does not contain the answer (for example, that it cannot find that information and the customer should contact customer support at support@marloweandfinch.com)

4. Set constraints on tone and length so the answers sound like the support team

Task: Create a variable called `rag_instruction` and assign it a prompt template string. The template must include the exact placeholders `{context}` and `{question}` so it can be used with `PromptTemplate.from_template()` later. Write the instruction text yourself, following the four requirements above.

Tip: Use triple quotation marks (`"""`) to write a multi-line string.

```
[8]: ### BEGIN SOLUTION

# --- Solution (rag_instruction pattern from Build Your First RAG System
    ↪activity, tailored to the Marlowe & Finch business context) ---
rag_instruction = """
You are a customer support assistant for Marlowe & Finch, an outdoor gear
    ↪company.
Use only the provided context to answer the customer's question.
If the answer is not in the context, say "I'm not sure how to answer that based
    ↪on our customer support documentation.
Please reach out to our team at support@marloweandfinch.com and we'll get back
    ↪to you."
Keep your answers concise (2-4 sentences) and friendly.

Question: {question}
Context: {context}
Answer:
"""

### END SOLUTION
```

The cell below builds a `PromptTemplate` object from the string you wrote and prints the input variables it detected. You should see `['context', 'question']`.

```
[9]: # Do not remove or edit this cell

prompt = PromptTemplate.from_template(rag_instruction)
print("PromptTemplate input variables:", prompt.input_variables)
```

PromptTemplate input variables: `['context', 'question']`

1.5 Part 4. Assemble the Baseline RAG Chain

Now you will assemble the complete RAG chain that you built in the Build Your First RAG System activity. The components are the same; only the corpus, the prompt, and the queries are different.

First, initialize the LLM.

```
[10]: # Do not remove or edit this cell

llm = ChatOpenAI(model="gpt-4o")
```

Task: Write a function called `format_docs` that takes a list of `Document` objects and returns a single string containing their `page_content`, separated by two newlines (`\n\n`).

```
[11]: ### BEGIN SOLUTION
# --- Solution (format_docs pattern from LangChain Primer and Build Your First
#      ↪ RAG System activities) ---
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

### END SOLUTION
```

Task: Assemble the baseline RAG chain. Save it to a variable called `rag_chain`. The chain should use: - A dictionary mapping where "context" runs the retriever and pipes its output through `format_docs`, and "question" uses `RunnablePassthrough()` - The prompt object created in Part 3 - The llm object initialized above - A `StrOutputParser()` at the end

Use the pipe operator (`|`) to connect the components.

```
[12]: ### BEGIN SOLUTION

# --- Solution (full RAG chain pattern from Build Your First RAG System
#      ↪ activity) ---
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

### END SOLUTION
```

Now let's test the chain on three customer queries. These are written to feel like what real shoppers actually type into a chat box. Pay attention to how the chain handles each one. They are deliberately different from each other:

- **Query 1** is short and underspecified, the kind of thing someone types when they have a quick question in their head.
- **Query 2** is a specific, scenario-based question about a real Marlowe & Finch policy.
- **Query 3** is a specific question that tests a particular edge case in Marlowe & Finch's policies.

```
[13]: # Do not remove or edit this cell

baseline_queries = [
    "is the tent waterproof",
    "can i return a tent i used on a weekend trip",
    "do you ship to australia"
```

```

]

baseline_answers = {}

for q in baseline_queries:
    answer = rag_chain.invoke(q)
    baseline_answers[q] = answer
    print(f"Q: {q}")
    print(f"A: {answer}\n")
    print("-" * 60)

```

Q: is the tent waterproof

A: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Q: can i return a tent i used on a weekend trip

A: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Q: do you ship to australia

A: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Task: Review the three answers above. In the markdown cell below, answer the following:

1. How did the chain handle each of the three queries? Be specific: which ones produced clean answers, and which ones had problems? Were there any queries that the chain should have been able to answer but did not answer correctly?
2. If a customer support manager saw these three answers, which one would they be most uncomfortable publishing on the website, and why?

<Double click this Markdown cell to make it editable, and record your findings here.>

1.6 Part 5. Add Query Rewriting and Compare

Short, casual queries are a known weakness of vector retrieval. A query like “is the tent waterproof” is short and underspecified compared to the full-sentence documentation in the knowledge base.

In the query transformation activity, you saw that one way to address this is to rewrite the query into a longer, more complete version before sending it to the retriever. In this part, you will add a query rewriting step in front of the baseline chain and compare the results.

Task: Create a variable named `rewrite_prompt_template` that contains a prompt for rewriting

a customer query. This prompt will be combined with the actual customer query using `.format()` in the next cell. Use the placeholder `{short_query}` where the customer's original query should appear.

Your prompt should:

1. Explain the task (rewrite a short customer query into a more complete version)
2. Make it clear that the rewritten query should preserve the customer's intent
3. Ask for exactly one rewritten query as output, not a list or numbered alternatives

Note: Because this template contains a curly-brace placeholder, define it as a regular string now and use `.format()` later, rather than as an f-string directly.

Tip: This is similar to the query rewriting work you did in the Improve RAG Retrieval Through Query Transformation activity, but there you asked for multiple alternatives. Here you want exactly one rewritten query so you can drop it straight into the existing retriever.

```
[14]: ### BEGIN SOLUTION

# --- Solution (rewrite prompt pattern from Improve RAG Retrieval Through Query
↳ Transformation activity; .format() pattern for templates with placeholders
↳ from Lab 7's validation prompt) ---
rewrite_prompt_template = """
You are an expert at rewriting short customer support queries into complete,
well-formed questions that preserve the original intent.

Original query: {short_query}

Rewrite this query as a single complete question. Return only the rewritten
question, with no preamble, numbering, or alternatives.
"""

### END SOLUTION
```

The cell below uses your `rewrite_prompt_template` to rewrite each of the three queries from Part 4. To keep things fast and consistent, we rewrite each query once and save the results in a dictionary called `rewritten_queries`. That way, when we compare baseline answers to rewritten-query answers, we are using the same rewrites throughout.

```
[15]: # Do not remove or edit this cell

rewritten_queries = {}

for q in baseline_queries:
    filled_prompt = rewrite_prompt_template.format(short_query=q)
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": filled_prompt}]
    )
```

```
rewritten_queries[q] = response.choices[0].message.content.strip()
print(f"Original: {q}")
print(f"Rewritten: {rewritten_queries[q]}\n")
```

Original: is the tent waterproof
 Rewritten: Is the tent waterproof?

Original: can i return a tent i used on a weekend trip
 Rewritten: Can I return a tent that I used on a weekend trip?

Original: do you ship to australia
 Rewritten: Do you offer shipping to Australia?

Now let's run the full pipeline (the rewritten query through the existing RAG chain) and compare the answers side by side with the baseline.

```
[16]: # Do not remove or edit this cell

for q in baseline_queries:
    rewritten = rewritten_queries[q]
    rewritten_answer = rag_chain.invoke(rewritten)

    print(f"Original query: {q}")
    print(f"Rewritten query: {rewritten}\n")
    display(Markdown(f"**Baseline answer:** {baseline_answers[q]}"))
    display(Markdown(f"**Rewritten-query answer:** {rewritten_answer}"))
    print("=" * 60)
```

Original query: is the tent waterproof
 Rewritten query: Is the tent waterproof?

Baseline answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Rewritten-query answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

```
=====
Original query: can i return a tent i used on a weekend trip
Rewritten query: Can I return a tent that I used on a weekend trip?
```

Baseline answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Rewritten-query answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

=====

Original query: do you ship to australia
Rewritten query: Do you offer shipping to Australia?

Baseline answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

Rewritten-query answer: I'm not sure how to answer that based on our customer support documentation. Please reach out to our team at support@marloweandfinch.com and we'll get back to you.

=====

Task: Review the side-by-side comparison above. In the markdown cell below, answer the following:

1. On which of the three queries did query rewriting visibly change the answer? Did it improve the answer, make it worse, or leave it about the same?
2. Look at the rewritten queries themselves. Are they faithful to the original intent, or did the rewriting step change what the customer was asking about?
3. Query rewriting adds an extra LLM call before every retrieval, which costs both money and a bit of latency. Based on what you saw, is the improvement worth the added cost for this use case? When would you turn it on, and when would you leave it off?

<Double click this Markdown cell to make it editable, and record your findings here.>

1.7 Part 6. Analysis

You have now built a complete RAG assistant for Marlowe & Finch and tested a query rewriting improvement on top of it. In this section, reflect on the system as a whole.

Answer the following questions in the markdown cell below:

1. **Explaining the system to the team:** Marlowe & Finch's customer support manager is not technical. They want to know, in plain language, why the assistant sometimes gives a great answer and sometimes gives a vague one. Using what you observed in this lab, write a short explanation (3-5 sentences) you could give the manager. Avoid complicated technical jargon. Do not assume they know what an embedding is.
2. **Biggest deployment risk:** Marlowe & Finch is considering putting this assistant live on the website as a first-line responder to customer questions. Based on what you observed in this lab, what is the single biggest risk of deploying this prototype as-is? Name one specific type of customer question that would likely cause the assistant to behave badly, and explain what you would recommend to the team before going live.

<Double click this Markdown cell to make it editable, and record your findings here.>

1.8 Part 7. Reflection: AI Usage

1. Did you use AI tools for this lab? If yes, which ones and at what points in your work? If no, briefly explain your reasoning.

2. If you used AI, describe one specific prompt that was useful and explain why it worked. If you did not use AI, walk through one part of the lab where you had to figure something out on your own and explain how you got there.
3. How did you verify that your work was correct? What would you look for to catch a mistake, whether it came from AI or from your own reasoning?
4. What is one thing you would do differently next time, either in how you approached the lab or in how you used (or did not use) AI?

Record your findings in the cell below.

<Double click this Markdown cell to make it editable, and record your findings here.>